Below is a **master, industry-oriented table of Cloud & DevOps design patterns**, covering **what the pattern is**, **when to use it**, and **real software / cloud services that implement it in practice**.

This table is aligned with **real-world cloud architecture, DevOps pipelines, SRE practices, and production systems**, not just textbook definitions.

☁ Cloud & DevOps Design Patterns — Master Table

#	Pattern Name	Category	What Problem It Solves	Typical Use Case	Real Software / Cloud Services Using It
1	**Microservices Pattern**	Architecture	Monolith scalability & agility	Independent service deployment	Netflix, Amazon, Uber
2	**API Gateway Pattern**	Integration	Centralized API management	Auth, rate limiting, routing	AWS API Gateway, Kong, NGINX
3	**Circuit Breaker**	Resilience	Prevent cascading failures	Handle downstream failures	Netflix Hystrix, Resilience4j
4	**Retry Pattern**	Resilience	Temporary failure recovery	Network / API retries	Spring Retry, AWS SDK
5	**Bulkhead Pattern**	Resilience	Resource isolation	Prevent total system crash	Netflix, Istio
6	**Service Mesh Pattern**	Networking	Service-to-service communication	Traffic control, mTLS	Istio, Linkerd
7	**Sidecar Pattern**	Deployment	Cross-cutting concerns	Logging, monitoring, proxy	Kubernetes, Envoy
8	**Immutable Infrastructure**	Infrastructure	Configuration drift	Predictable deployments	AWS AMI, HashiCorp Packer
9	**Infrastructure as Code (IaC)**	DevOps	Manual infra errors	Version-controlled infra	Terraform, AWS CloudFormation

10	**Blue-Green Deployment**	Deployment	Zero-downtime release	Instant rollback	AWS, Azure DevOps
11	**Canary Deployment**	Deployment	Risky releases	Gradual traffic shift	Google, Spinnaker
12	**Rolling Deployment**	Deployment	Downtime avoidance	Sequential pod updates	Kubernetes
13	**Event-Driven Architecture**	Architecture	Loose coupling	Async processing	Apache Kafka, AWS SNS/SQS
14	**CQRS**	Data	Read/write optimization	High-scale systems	Axon, Microsoft
15	**Event Sourcing**	Data	Audit & replay	Financial systems	Axon, EventStore
16	**Strangler Fig Pattern**	Migration	Legacy modernization	Gradual monolith breakup	Amazon
17	**Cache-Aside Pattern**	Performance	Slow data access	Reduce DB load	Redis, AWS ElastiCache
18	**Write-Through Cache**	Performance	Consistent cache	Financial data	Hazelcast
19	**Leader Election Pattern**	Coordination	Single active instance	Distributed jobs	Apache Zookeeper
20	**Health Check Pattern**	Reliability	Detect unhealthy services	Auto-healing	Kubernetes, AWS ELB
21	**Auto-Scaling Pattern**	Scalability	Traffic spikes	Cost-efficient scaling	AWS ASG, Azure Scale Sets
22	**Failover Pattern**	Availability	Node failure	High availability	AWS RDS Multi-AZ
23	**DevOps CI/CD Pipeline**	DevOps	Manual builds	Fast delivery	Jenkins, GitHub Actions
24	**Observability Pattern**	Monitoring	Production visibility	Metrics & tracing	Prometheus, Grafana
25	**Secrets Management Pattern**	Security	Credential leaks	Secure secrets	HashiCorp Vault, AWS Secrets Manager

26	**Zero Trust Architecture**	Security	Network trust issues	Secure access	Google BeyondCorp
27	**GitOps Pattern**	DevOps	Config drift	Declarative ops	Argo CD, Flux
28	**Multi-Region Deployment**	Availability	Regional outages	Global apps	AWS, Cloudflare
29	**Saga Pattern**	Transactions	Distributed TX failure	Microservice consistency	Camunda
30	**Chaos Engineering Pattern**	Resilience	Unknown failures	System hardening	Netflix Chaos Monkey

---

How to Use This Table Practically

- **Cloud Architects** → Architecture + Resilience patterns
- **DevOps Engineers** → CI/CD, GitOps, IaC, Deployment patterns
- **SREs** → Observability, Chaos, Auto-Scaling
- **Backend Developers** → Microservices, Saga, CQRS

If you want next:

- **One-page printable cheat sheet**
- **Pattern → Anti-pattern → Fix mapping**
- ☒ **Kubernetes-specific pattern table**
- **Banking-grade cloud reference architecture**

Just tell me which one you want.

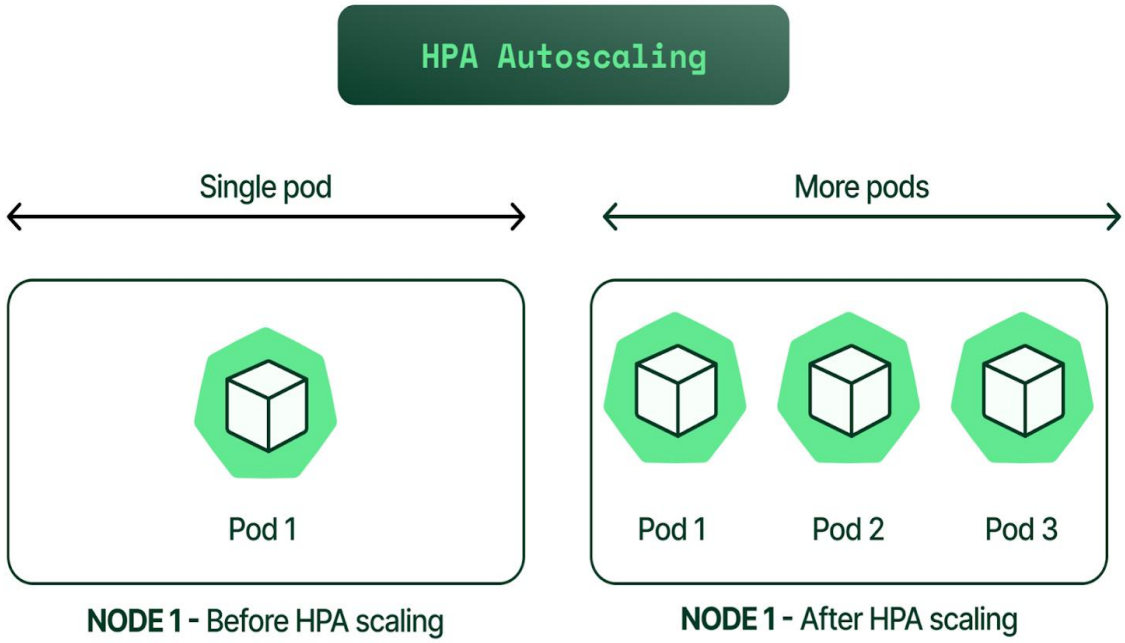
Perfect — below is a **production-grade, interview-ready, and architect-level explanation** of all three items you asked for.
This is exactly how **real cloud architects, DevOps leads, and banking SRE teams** think.

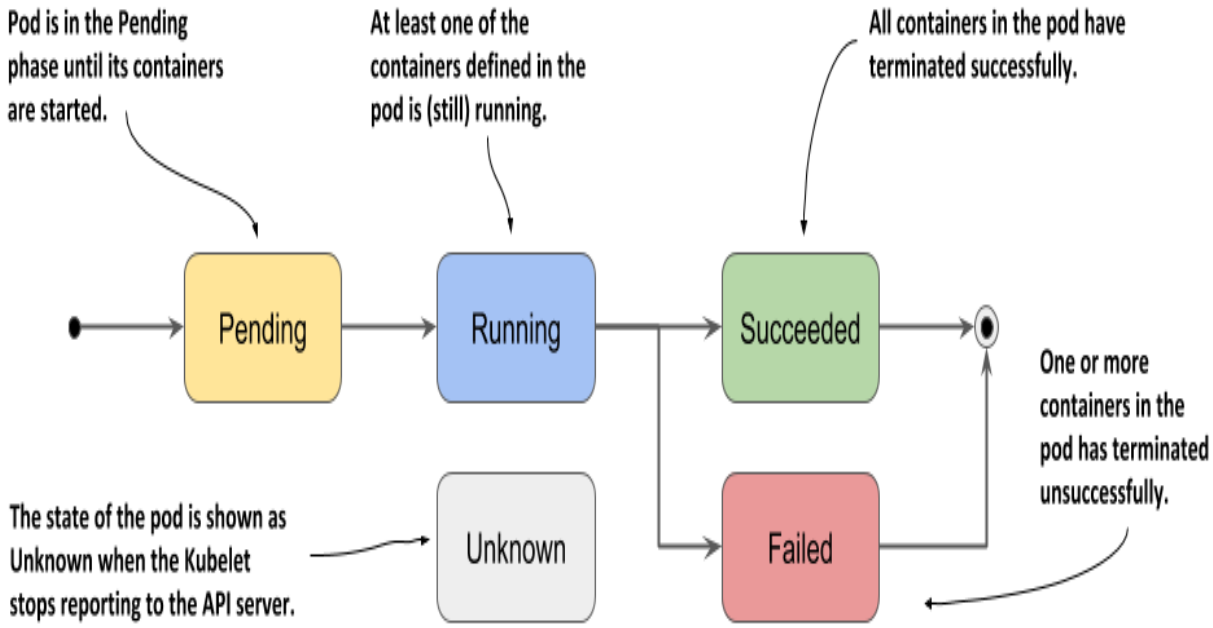
▯ Pattern → Anti-Pattern → Fix (Master Mapping Table)

#	Cloud / DevOps Pattern	Common Anti-Pattern	Why It Fails in Production	Correct Fix (What to Do Instead)
1	Microservices	Distributed Monolith	Tight DB & sync coupling	Domain-based services + async events
2	API Gateway	Client talks to all services	Security + chaos	Central API Gateway
3	Circuit Breaker	Infinite retries	Cascading failures	Circuit Breaker + Retry + Timeout
4	Auto Scaling	Manual server scaling	Slow, costly	HPA / ASG
5	Stateless Services	Session stored in memory	Scaling breaks	External session store
6	Immutable Infra	In-place patching	Config drift	Replace servers, don't modify
7	IaC	Manual infra creation	Non-reproducible	Terraform / CloudFormation
8	Canary Deployment	Big-bang release	Mass outage	Gradual traffic rollout
9	Observability	Logs only	Blind production	Metrics + Logs + Traces
10	Event-Driven	Chatty sync calls	Latency	Async events
11	GitOps	Manual kubectl changes	Drift	Git = single source of truth
12	Secrets Mgmt	Secrets in config files	Security breach	Vault / Secrets Manager
13	Saga	2PC transactions	Deadlocks	Compensating actions
14	Multi-AZ	Single AZ	Regional outage	Active-active AZs

15	Health Checks	Assume service is healthy	Silent failure	Liveness + Readiness

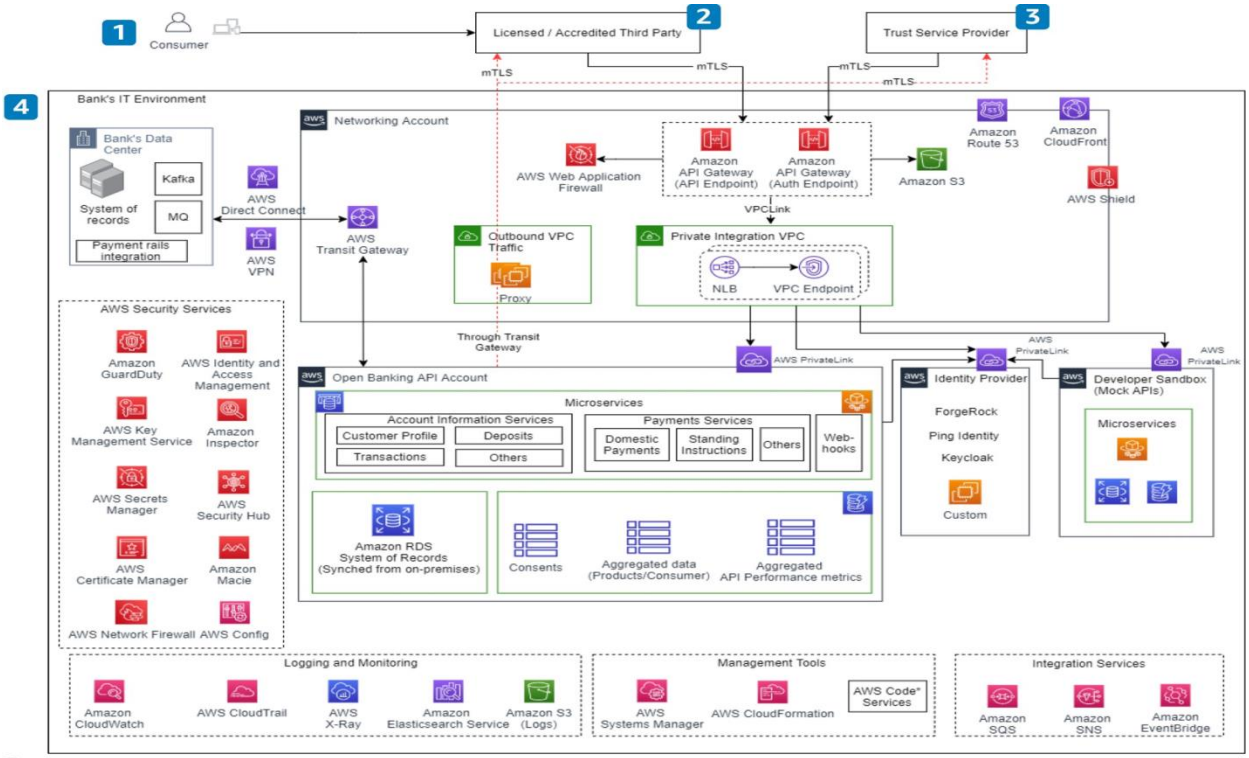
☒ Kubernetes-Specific Design Patterns (Production Table)

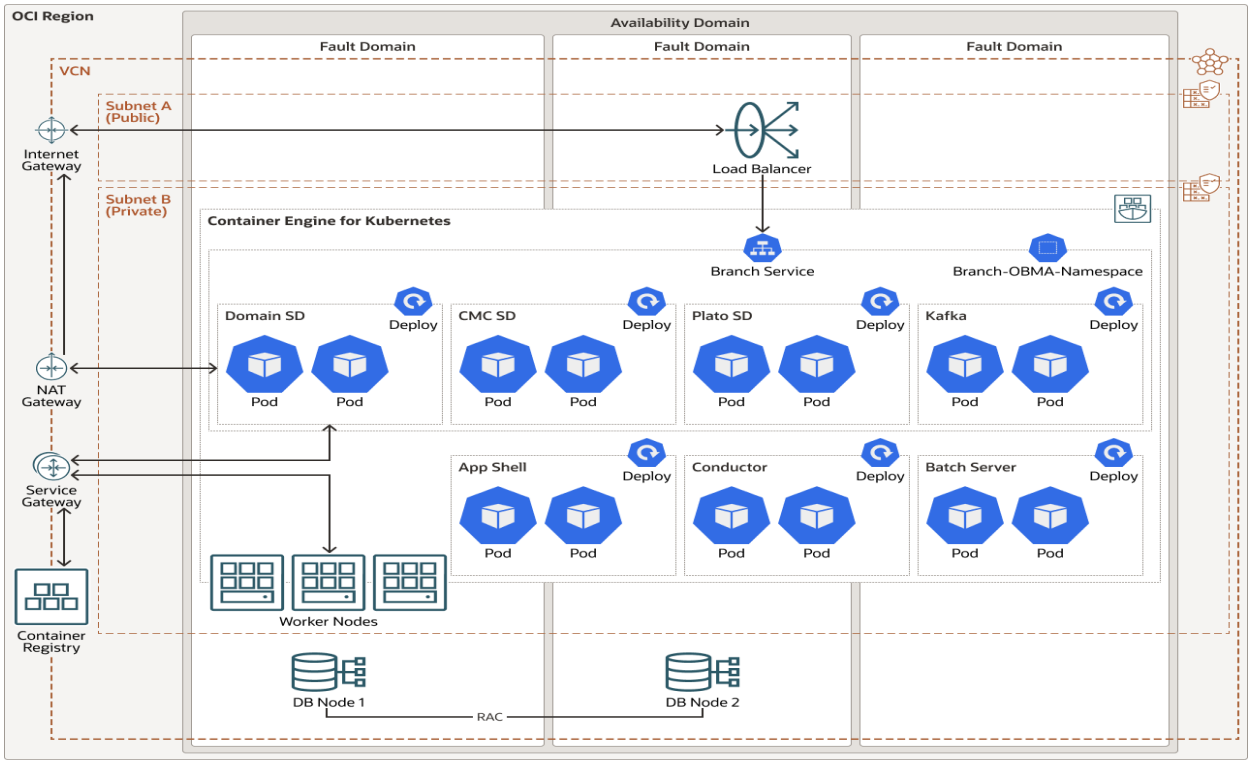


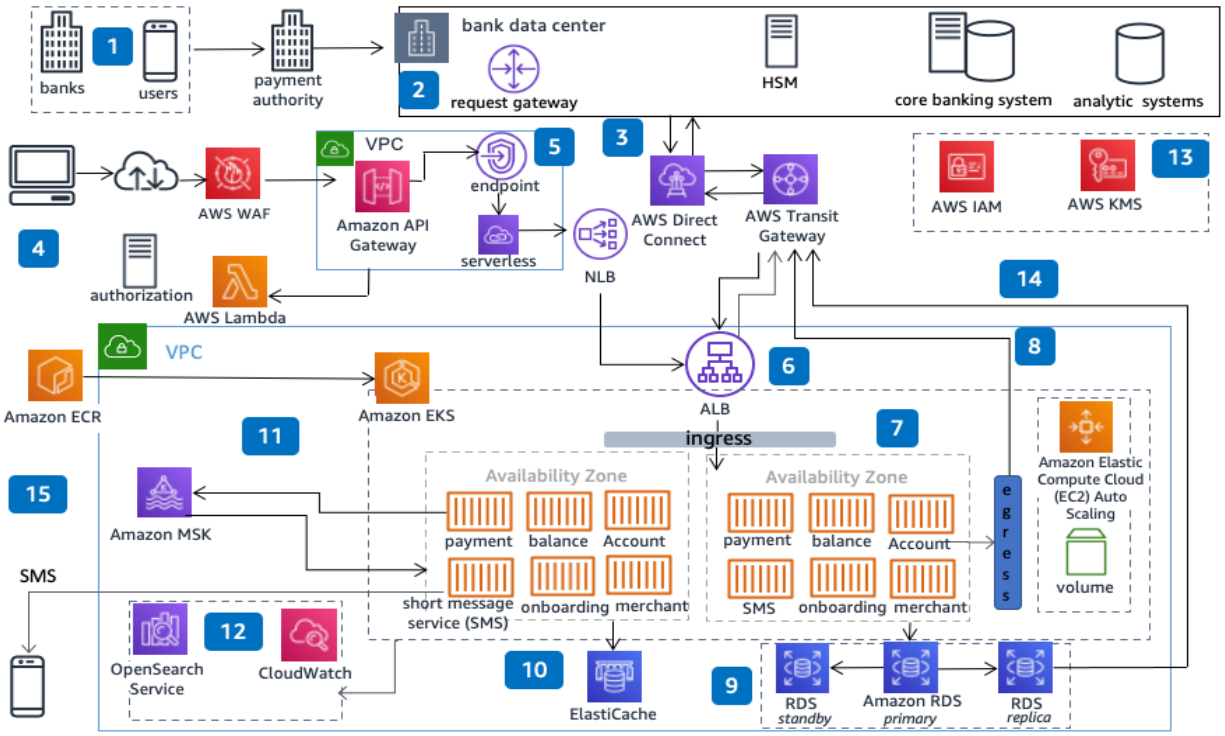


#	Kubernetes Pattern	Purpose	Real Usage
1	Pod Pattern	Smallest deployable unit	All workloads
2	Sidecar	Logging, proxy, security	Istio, Envoy
3	Init Container	Pre-startup checks	DB readiness
4	Rolling Update	Zero-downtime deploy	Default Deployment
5	Blue-Green	Instant rollback	Banking apps
6	Canary	Safe rollout	Payments
7	HPA	CPU/Memory scaling	Traffic spikes
8	Operator	Automate ops logic	Prometheus Operator
9	Service Mesh	mTLS, retries	Istio
10	ConfigMap	External config	App configs
11	Secret	Secure credentials	DB passwords
12	Liveness Probe	Restart stuck pods	Self-healing
13	Readiness Probe	Traffic control	Load balancers
14	DaemonSet	Node-level services	Logging agents
15	Job / CronJob	Batch processing	Reports
16	Namespace	Multi-tenancy	Dev / QA / Prod
17	Ingress	External routing	NGINX
18	StatefulSet	Stateful apps	Databases

🏦 Banking-Grade Cloud Reference Architecture (Real-World)







▯ High-Level Layers

Layer	Components	Why Banks Need It
Channel	Mobile, Web, ATM	Omni-channel access
Edge	WAF, CDN, DDoS	Internet security
Access	API Gateway, IAM	Strong auth
App	Microservices	Fast change
Integration	ESB, Kafka	Async & reliability
Data	RDBMS, Cache	ACID + speed
Security	HSM, Vault	Compliance
Ops	CI/CD, Monitoring	Zero downtime
DR	Multi-Region	Regulatory

🔐 Security & Compliance (Non-Negotiable)

Requirement	Implementation
Encryption at rest	KMS / HSM
Encryption in transit	mTLS
Zero Trust	Per-service auth
Audit logs	Immutable storage
RBAC	Least privilege
Compliance	PCI-DSS, ISO, SOC

🧰 Typical Banking Tech Stack

Area	Tools
API Gateway	Kong / Apigee
Containers	Kubernetes
Messaging	Apache Kafka
CI/CD	Jenkins
Secrets	HashiCorp Vault
Monitoring	Prometheus + Grafana
DB	Oracle / PostgreSQL
Cache	Redis
Cloud	AWS / Azure

🧠 Why This Matters (Reality Check)

✔These **patterns are asked in senior interviews**

✔These **decide production stability**

✔These **separate real architects from slide-ware engineers**